Bézier Curve + Curvature Comb

Generic 2-D / 3-D — GLMakie

Code and Plot Explanation for Julia Beginners

`34-bezier-curvature.jl`

E.R. Nurwijayadi

2026-03-15

Directed and curated by E.R. Nurwijayadi. Code and explanations AI-generated.

This document explains `34-bezier-curvature.jl`. It extends `31-bezier-generic.jl` with one major addition: the **curvature comb** — a visual representation of how much the curve bends at each point. New concepts: derivative control points, the curvature formula, the curvature normal vector in 2-D and 3-D, automatic comb scaling via quantile, and the `draw_comb!` function. The `draw_and_show` body and all drawing layers are otherwise identical to script 31.

Contents

1 The Two Plots **3**
1.1 Case 1 — Quadratic 2-D with Curvature Comb 3
1.2 Case 2 — Quartic 3-D with Curvature Comb 4

2 What Is Curvature? **4**

3 New Imports **5**

4 The Extended `BezierCurve` Struct **5**
4.1 Two new fields: `d1_pts` and `d2_pts` 5
4.2 `diff(points, dims=1)` — the derivative shortcut 6

5 The Math Functions **6**
5.1 `eval_poly` — shared evaluator 6
5.2 `bz_d1` and `bz_d2` 7

5.3 `bz_curvature` — the curvature formula 7

6 `draw_comb!` — Drawing the Curvature Comb 8

6.1 Naming convention: `!` means mutates the axis 8
6.2 Subsampling with integer division (`\div`) 9
6.3 `similar(base)` — allocating an output matrix 9
6.4 `LinearAlgebra` and `Statistics` functions used 9
6.4.1 `norm(v)` — vector magnitude 9
6.4.2 `cross(a, b)` — vector cross product 9
6.4.3 `quantile(data, p)` — statistical threshold 10
6.5 The 2-D normal vector — four steps 10
6.6 The 3-D principal normal vector 11
6.7 Drawing spikes and envelope 11

7 Auto-Scaling the Comb 12

8 What Is Identical to Script 31 13

9 Type Map 13

10 Entry Point 13

1. The Two Plots

1.1. Case 1 — Quadratic 2-D with Curvature Comb

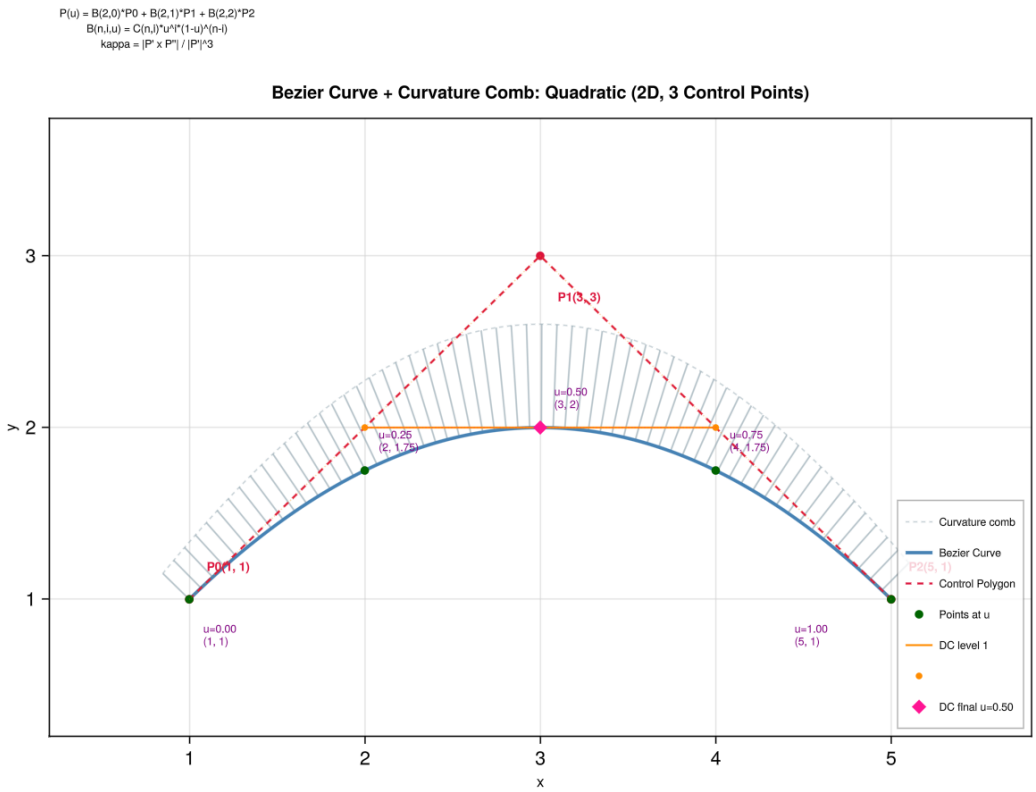


Figure 1: Case 1: quadratic Bézier with curvature comb in 2-D. The grey fan of spikes above the curve shows that curvature is highest near the centre and falls off symmetrically toward the endpoints.

1.2. Case 2 — Quartic 3-D with Curvature Comb

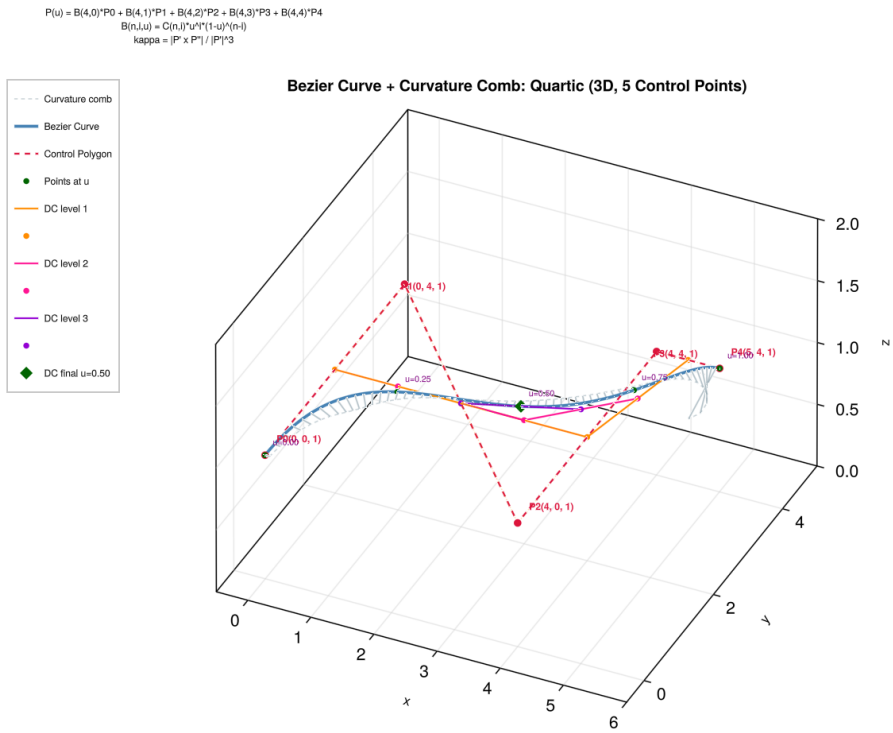


Figure 2: Case 2: quartic Bézier with curvature comb in 3-D. The grey spikes point along the principal normal of the curve at each sample point.

2. What Is Curvature?

Before reading the code, understand what is being measured.

The **curvature** κ of a curve at a point measures how sharply the curve bends there. A straight line has $\kappa = 0$. A circle of radius r has constant $\kappa = 1/r$. A tighter bend (smaller radius of curvature) gives a larger κ.

For a parametric curve $P(u)$ with first derivative $P'(u)$ and second derivative $P''(u)$, curvature is:

$$\kappa = \frac{|P' \times P''|}{|P'|^3}$$

In 2-D the cross product produces a scalar (the z-component of the 3-D cross product), so $|P' \times P''| = |P'_x P''_y - P'_y P''_x|$. In 3-D it produces a vector, and we take its norm.

The **curvature comb** is a standard visualisation technique in computer-aided design. At each point on the curve, a spike is drawn perpendicular to the tangent, with length proportional to κ. The envelope connecting the spike tips reveals how smoothly curvature changes along the curve.

Reading the 2-D plot: The spikes above the curve are tallest near $u = 0.5$ (the peak of the arc, where bending is greatest) and shrink symmetrically toward the endpoints. The quadratic Bézier has *constant* curvature for this symmetric configuration, so the comb tips trace a nearly uniform arc — the envelope is smooth and even.

3. New Imports

```
using LinearAlgebra
using Statistics
```

Two new standard library packages:

`LinearAlgebra`: provides `norm` (vector magnitude) and `cross` (3-D cross product). Both are used in curvature computation.

`Statistics`: provides `quantile`, used to auto-scale the comb height. These are part of Julia's standard library — no `Pkg.add` needed.

4. The Extended `BezierCurve` Struct

```
struct BezierCurve
    points  ::Matrix{Float64}
    n       ::Int
    degree  ::Int
    dim     ::Int
    d1_pts  ::Matrix{Float64}          # (n-1 x dim)
    d2_pts  ::Union{Matrix{Float64}, Nothing}  # (n-2 x dim)
end

function BezierCurve(points::Matrix{Float64})
    n      = size(points, 1)
    deg    = n - 1
    dim    = size(points, 2)
    d1_pts = deg .* diff(points, dims=1)
    d2_pts = size(d1_pts, 1) > 1 ?
             (deg - 1) .* diff(d1_pts, dims=1) : nothing
    BezierCurve(points, n, deg, dim, d1_pts, d2_pts)
end
```

4.1. Two new fields: `d1_pts` and `d2_pts`

These store the **derivative control points** — the control points of the first and second derivative curves.

For a Bézier curve of degree n with control points $P_0, \ldots, P_n$, the first derivative is itself a Bézier curve of degree $n-1$ with control points:

$$Q_i = n \cdot (P_{i+1} - P_i), \quad i = 0, \ldots, n-1$$

The second derivative is a Bézier of degree $n-2$ with control points derived similarly from Q_i.

This means the derivative at any u can be evaluated using the same Bernstein machinery — just with different control points and degree.

`d2_pts ::Union{Matrix{Float64}, Nothing}`: For a linear curve (degree 1, two control points), `d1_pts` has one row — a single point, not a curve — and there is no second derivative in the same sense. `Union{Matrix{Float64}, Nothing}` means the field can hold either a matrix *or* the special value `nothing`. This is Julia's way of expressing an optional value.

4.2. `diff(points, dims=1)` — the derivative shortcut

```
d1_pts = deg .* diff(points, dims=1)
```

`diff(A, dims=1)` computes differences between consecutive rows of matrix `A`. For a 5×3 matrix it returns a 4×3 matrix where row i is `A[i+1,:] - A[i,:]`.

Multiplied by `deg` (broadcast `.*`), this gives exactly the first derivative control points $Q_i = n(P_{i+1} - P_i)$.

For Case 1 (quadratic 2-D):

$$\texttt{points} = \begin{bmatrix} 1 & 1 \\ 3 & 3 \\ 5 & 1 \end{bmatrix}, \quad \texttt{deg = 2}$$

$$\texttt{diff(points, dims=1)} = \begin{bmatrix} 2 & 2 \\ 2 & -2 \end{bmatrix}$$

$$\texttt{d1_pts = 2 .* ...} = \begin{bmatrix} 4 & 4 \\ 4 & -4 \end{bmatrix}$$

So $Q_0 = [4, 4]$ and $Q_1 = [4, -4]$.

$$\texttt{diff(d1_pts, dims=1)} = \begin{bmatrix} 0 & -8 \end{bmatrix}$$

$$\texttt{d2_pts = 1 .* ...} = \begin{bmatrix} 0 & -8 \end{bmatrix} \quad \text{(one row — degree 0 constant)}$$

This means $P''(u) = [0, -8]$ for all u — constant second derivative, confirming that a quadratic Bézier has constant curvature under symmetric conditions.

5. The Math Functions

5.1. `eval_poly` — shared evaluator

```
function eval_poly(pts::Matrix{Float64}, u::Float64)
    n = size(pts, 1) - 1
    sum(bernstein(n, i-1, u) .* pts[i,:] for i in 1:size(pts,1))
end
```

A generalised version of `bz_point` from script 31. It evaluates a Bézier curve defined by *any* control point matrix at parameter `u`, computing the degree from the number of rows. This makes it reusable for the original curve (`bz.points`), the first derivative (`bz.d1_pts`), and the second derivative (`bz.d2_pts`).

5.2. bz_d1 and bz_d2

```
function bz_d1(bz::BezierCurve, u::Float64)
    eval_poly(bz.d1_pts, u)
end

function bz_d2(bz::BezierCurve, u::Float64)
    isnothing(bz.d2_pts) ? zeros(bz.dim) : eval_poly(bz.d2_pts, u)
end
```

`bz_d1` evaluates the first derivative $P'(u)$ by calling `eval_poly` on the derivative control points.

`bz_d2` handles the `nothing` case: if `d2_pts` is `nothing` (for a linear curve), it returns a zero vector, which makes curvature zero everywhere — correct, since a straight line has no bending.

`isnothing(x)` returns `true` if `x` is `nothing`.

For Case 1 at $u = 0.5$:

`d1_pts` $= \begin{bmatrix} 4 & 4 \\ 4 & -4 \end{bmatrix}$, degree 1.

`eval_poly(d1_pts, 0.5)` evaluates a linear Bézier with these two points: $0.5 \times [4, 4] + 0.5 \times [4, -4] = [4, 0]$

So $P'(0.5) = [4, 0]$ — the tangent at the peak is horizontal. ✓

`d2_pts` $= \begin{bmatrix} 0 & -8 \end{bmatrix}$, degree 0.

`eval_poly(d2_pts, 0.5)` $= [0, -8]$

So $P''(0.5) = [0, -8]$ for all u. ✓

5.3. bz_curvature — the curvature formula

```
function bz_curvature(bz::BezierCurve, u::Float64)
    d1    = bz_d1(bz, u)
    d2    = bz_d2(bz, u)
    speed = norm(d1)
    speed < 1e-10 && return 0.0
    if bz.dim == 2
        abs(d1[1]*d2[2] - d1[2]*d2[1]) / speed^3
    else
        norm(cross(d1, d2)) / speed^3
    end
end
```

`speed = norm(d1)`: The speed is the magnitude of the velocity vector $P'(u)$. It appears as $|P'|$ in the denominator of the curvature formula.

`speed < 1e-10 && return 0.0`: A short-circuit guard: if the curve has zero speed (a cusp or stationary point), curvature is undefined. Returning 0.0 prevents division by zero. The `&&` operator evaluates the right side only if the left side is `true` — a concise way to write `if speed < 1e-10; return 0.0; end`.

2-D branch: The curvature formula uses the 2-D scalar cross product $|P'_x P''_y - P'_y P''_x|$, the z-component of the 3-D cross product when both vectors lie in the xy-plane.

3-D branch: `cross(d1, d2)` computes the 3-D cross product vector. `norm(...)` takes its magnitude. This is the standard formula $\kappa = |P' \times P''|/|P'|^3$.

Curvature at $u = 0.5$ for Case 1:

$P'(0.5) = [4, 0], \quad P''(0.5) = [0, -8]$

$\text{speed} = \sqrt{4^2 + 0^2} = 4$

$|P'_x P''_y - P'_y P''_x| = |4 \times (-8) - 0 \times 0| = 32$

$\kappa(0.5) = \dfrac{32}{4^3} = \dfrac{32}{64} = 0.5$

This means the radius of curvature at the peak is $1/0.5 = 2$ data units — a fairly tight bend for a curve spanning 4 units horizontally.

6. `draw_comb!` — Drawing the Curvature Comb

```
function draw_comb!(ax, bz::BezierCurve, u_vals, curve_pts,
                    kappas, comb_scale, is_3d)
    step  = max(1, length(u_vals) ÷ 60)
    u_sub = u_vals[1:step:end]
    m     = length(u_sub)

    base = hcat([bz_point(bz, u) for u in u_sub]...)'
    d1v  = hcat([bz_d1(bz, u)   for u in u_sub]...)'
    d2v  = hcat([bz_d2(bz, u)   for u in u_sub]...)'
    kap  = [bz_curvature(bz, u) for u in u_sub]
    tips = similar(base)
    ...
end
```

6.1. Naming convention: `!` means mutates the axis

The function is named `draw_comb!` with an exclamation mark. In Julia, the `!` suffix is a naming convention indicating that the function *mutates* one of its arguments — here, it adds plot elements to `ax` in place. This mirrors Makie's own `lines!`, `scatter!`, `text!` convention. The `!` is not required by the language; it is a community convention that signals mutation.

6.2. Subsampling with integer division (\div)

```
step  = max(1, length(u_vals) ÷ 60)
u_sub = u_vals[1:step:end]
```

Drawing one spike per curve sample (300 spikes) would produce a dense, unreadable comb. `length(u_vals) \div 60` performs **integer division** (the `\div` operator, typed as
+ `div` + Tab in Julia) to calculate a step size that gives roughly 60 evenly spaced spikes. `max(1, ...)` ensures the step is at least 1. `u_vals[1:step:end]` slices the array with a stride of `step`.

6.3. `similar(base)` — allocating an output matrix

```
tips = similar(base)
```

`similar(A)` allocates a new matrix with the same shape and element type as `A`, but with *uninitialised* values. It is equivalent to `zeros(size(base,1), size(base,2))` but slightly more efficient since it skips the zero-fill step. `tips` will hold the endpoint of each curvature spike.

6.4. `LinearAlgebra` and `Statistics` functions used

6.4.1. `norm(v)` — vector magnitude

`norm(v)` computes the Euclidean length of a vector:

$$\|\mathbf{v}\| = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2}$$

In the comb code it appears as `norm(d1v[i,:])` (the speed $\|P'(u)\|$) and `norm(N_raw)` (the length of the raw principal normal vector). It also appears in `bz_curvature` as `norm(d1)` for the speed denominator and `norm(cross(d1, d2))` for the cross product magnitude.

6.4.2. `cross(a, b)` — vector cross product

`cross(a, b)` computes the 3-D cross product of two vectors using the 3×3 determinant:

$$\mathbf{a} \times \mathbf{b} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \end{vmatrix} = \begin{bmatrix} a_2 b_3 - a_3 b_2 \\ a_3 b_1 - a_1 b_3 \\ a_1 b_2 - a_2 b_1 \end{bmatrix}$$

The result is a vector *perpendicular* to both $\mathbf{a}$ and $\mathbf{b}$, with magnitude:

$$\|\mathbf{a} \times \mathbf{b}\| = \|\mathbf{a}\| \, \|\mathbf{b}\| \sin\theta$$

where θ is the angle between them. This magnitude is used directly in the curvature formula:

```
kappa = norm(cross(d1, d2)) / speed^3
```

which corresponds to:

$$\kappa = \frac{\|P'(u) \times P''(u)\|}{\|P'(u)\|^3}$$

6.4.3. `quantile(data, p)` — statistical threshold

`quantile(data, p)` returns the value below which a fraction p of the data falls. `p = 0.95` gives the 95th percentile: the value that is larger than 95% of all observations.

```
kap_95 = quantile(kappas, 0.95)
```

Here it provides a robust upper reference for curvature — ignoring the top 5% of outlier values that can appear at curve endpoints where the formula is sensitive to small numerical errors. The comb is then scaled so that a spike at `kap_95` reaches 15% of the curve extent.

6.5. The 2-D normal vector — four steps

```
for i in 1:m
    spd = norm(d1v[i,:])
    spd = spd > 1e-10 ? spd : 1e-10
    T   = d1v[i,:] / spd        # unit tangent
    N   = [-T[2], T[1]]         # unit normal (90-degree rotation)
    tips[i,:] = base[i,:] .+ comb_scale * kap[i] .* N
end
```

Each iteration computes the spike tip in four steps.

Step 1 — Speed $\|\mathbf{v}\| = \|P'(u)\|$

$$\texttt{spd} = \|\mathbf{d1}\| = \sqrt{d1_x^2 + d1_y^2}$$

`norm(d1v[i,:])` returns the Euclidean length of the velocity vector at this sample point. The guard `spd > 1e-10 ? spd : 1e-10` prevents division by zero at a cusp.

Step 2 — Unit Tangent T

$$T = \frac{P'(u)}{\|P'(u)\|} \qquad \texttt{T = d1v[i,:] / spd}$$

Dividing the velocity vector by its own length produces a vector of length exactly 1 pointing in the direction of travel.

Step 3 — Unit Normal N

$$N = \begin{bmatrix} -T_y \\ T_x \end{bmatrix} \qquad \texttt{N = [-T[2], T[1]]}$$

Rotating a 2-D unit vector $[T_x, T_y]$ by 90° counter-clockwise gives $[-T_y, T_x]$. This is the unit normal vector perpendicular to the tangent, pointing toward the centre of curvature.

Step 4 — Tip position

$$\text{tip} = P(u) + s \cdot \kappa \cdot N \qquad \texttt{tips[i,:] = base[i,:] .+ comb_scale * kap[i] .* N}$$

The spike endpoint is displaced from the curve point by `comb_scale` $\times\ \kappa$ in the normal direction. A higher curvature produces a longer spike.

6.6. The 3-D principal normal vector

```
for i in 1:m
    d1i   = d1v[i,:]
    d2i   = d2v[i,:]
    N_raw = cross(cross(d1i, d2i), d1i)
    n_mag = norm(N_raw)
    N     = n_mag > 1e-10 ? N_raw / n_mag : zeros(3)
    tips[i,:] = base[i,:] .+ comb_scale * kap[i] .* N
end
```

In 3-D, the normal direction is the **principal normal**: the direction in which the curve is turning. The formula $N_{\text{raw}} = (P' \times P'') \times P'$ gives the component of P'' perpendicular to P', which points toward the centre of curvature. After normalising, this is the unit principal normal.

The inner cross product $P' \times P''$ uses the full 3×3 determinant formula. For $\mathbf{a} = [a_1, a_2, a_3]$ and $\mathbf{b} = [b_1, b_2, b_3]$:

$$\mathbf{a} \times \mathbf{b} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \end{vmatrix} = \begin{bmatrix} a_2 b_3 - a_3 b_2 \\ a_3 b_1 - a_1 b_3 \\ a_1 b_2 - a_2 b_1 \end{bmatrix}$$

The result is a vector perpendicular to both $\mathbf{a}$ and $\mathbf{b}$, with magnitude:

$$\|\mathbf{a} \times \mathbf{b}\| = \|\mathbf{a}\| \, \|\mathbf{b}\| \sin\theta$$

where θ is the angle between them. In Julia: `cross(d1i, d2i)`.

The outer cross product $(P' \times P'') \times P'$ then gives a vector that lies *in* the plane of P' and P'' but is perpendicular to P' — exactly the principal normal direction.

> **Why not just use P'' directly?** P'' has a component along the tangent (from changing speed) and a component perpendicular to it (from changing direction). Only the perpendicular component is relevant to curvature. The double cross product $(P' \times P'') \times P'$ projects out the tangential component, leaving only the normal component.

6.7. Drawing spikes and envelope

```
# Spike lines (one per subsampled point)
for i in 1:m
    lines!(ax, [base[i,1], tips[i,1]],
               [base[i,2], tips[i,2]],
               color=(COLOR_COMB, 0.65), linewidth=1.2)
end
# Envelope (connecting the tips)
lines!(ax, tips[:,1], tips[:,2],
    color=(COLOR_COMB, 0.5), linewidth=1.0,
    linestyle=:dash, label="Curvature comb")
```

Each spike is a two-point line from `base[i,:]` to `tips[i,:]`. The envelope connects all the tips in sequence, tracing the outline of the comb. Only the envelope carries a `label` (for the legend); the individual spikes are unlabelled.

`COLOR_COMB = RGBAf(0.565, 0.643, 0.682, 1.0)`: This colour is defined directly as an RGBA floating-point value instead of a named symbol. `RGBAf(r, g, b, a)` specifies red, green, blue, and alpha channels as floats in $[0, 1]$. The hex equivalent is approximately `#90A4AE` — a muted steel blue-grey that blends with the plot without competing with the curve colours.

7. Auto-Scaling the Comb

```
u_vals    = collect(LinRange(0.0, 1.0, n_pts))
curve_pts = bz_curve(bz, n_pts)
kappas    = [bz_curvature(bz, u) for u in u_vals]

ext      = maximum(maximum(curve_pts[:,1:2], dims=1) .-
                   minimum(curve_pts[:,1:2], dims=1))
kap_95   = quantile(kappas, 0.95)
comb_scale = kap_95 > 1e-10 ? 0.15 * ext / kap_95 : 0.1
```

The comb scale is computed so the spikes look proportionate regardless of the curve's size or how curved it is.

`ext`: the extent of the curve — the maximum of (width, height) in the xy-plane. This captures the overall size of the figure. `curve_pts[:,1:2]` slices only the x and y columns (ignoring z), so the extent is a 2-D measurement even for the 3-D curve.

`quantile(kappas, 0.95)`: The 95th-percentile curvature value. Using the maximum curvature directly would make the scale too sensitive to any curvature spikes at the endpoints (where the formula can amplify small errors). The 95th percentile ignores the top 5% of outliers and gives a robust representative "high curvature" value.

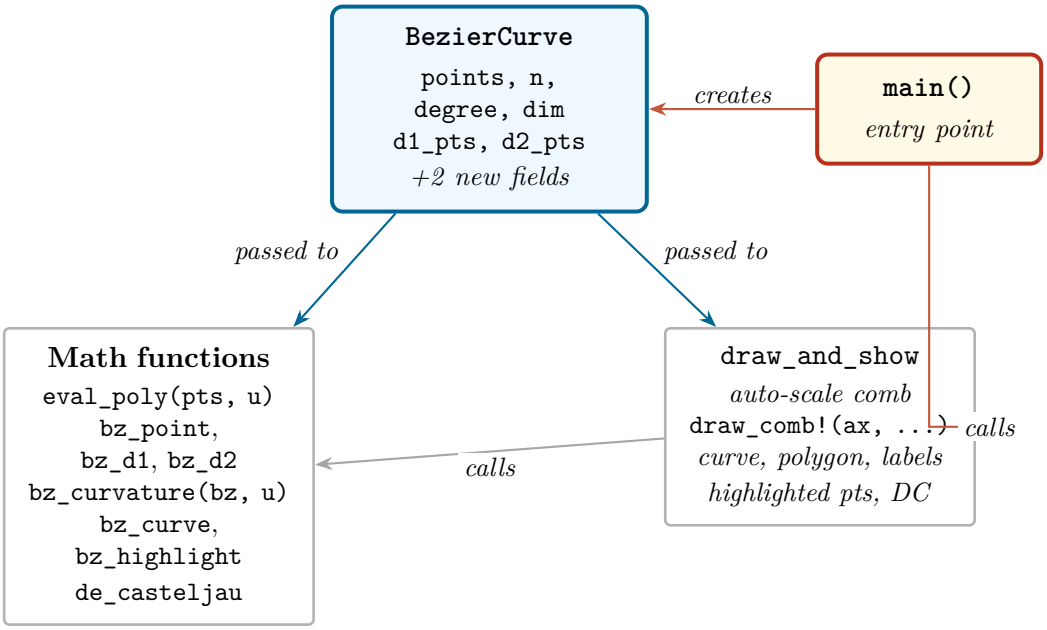
`comb_scale = 0.15 * ext / kap_95`: The spike for the 95th-percentile curvature will be 15% of the curve's extent — tall enough to be clearly visible without dominating the plot. All other spikes scale proportionally.

> **For Case 1 (quadratic 2-D):**
>
> `kappas` contains 300 curvature values. For the symmetric quadratic, curvature is approximately constant at ≈ 0.5.
>
> `ext` ≈ 4 (the curve spans $x = 1$ to 5, so width $= 4$).
>
> `kap_95` ≈ 0.5
>
> `comb_scale` $= 0.15 \times 4/0.5 = 1.2$
>
> Each spike at $\kappa = 0.5$ will have length $1.2 \times 0.5 = 0.6$ data units, which matches the visible spike height in the plot.

8. What Is Identical to Script 31

Component	Note
`CASE_DATA`	Identical (different output filenames).
`degree_name`, `dim_name`	Identical.
`de_casteljau`	Identical.
Colour constants	Identical except `COLOR_COMB` added as `RGBAf`.
`draw_and_show` body	Identical from control polygon onward. Comb is drawn *first* (so it sits under the curve).
Equation label	Same two lines; third line changed from `[evaluated via Symbolics.jl]` to `kappa = |P' x P"| / |P'|^3`.

9. Type Map



10. Entry Point

```julia
function main()
    data = CASE_DATA[CASE]
    bz   = BezierCurve(data.points)
    draw_and_show(bz, data.u_highlight, data.u_dc, data.output)
end

main()
```

How to run:

```
julia 34-bezier-curvature.jl
```

Change `const CASE = 1` for the quadratic 2-D plot, `const CASE = 2` for the quartic 3-D interactive view. Press Enter to close.

Explanation for `34-bezier-curvature.jl` | Directed and curated by E.R. Nurwijayadi | Code and explanations AI-generated